# 임베디드 시스템

## Embedded Systems

WiringPi | RGB LED | Active Buzzer | Switch

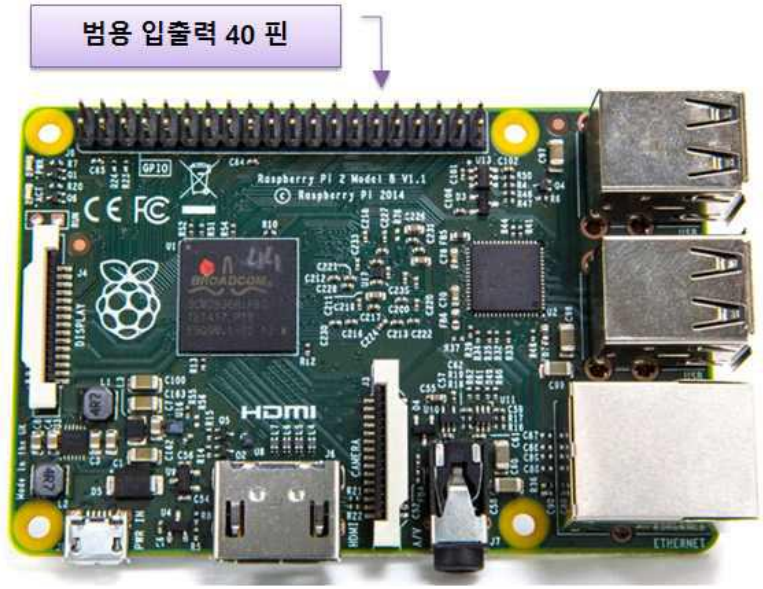
강원대학교 컴퓨터공학과 권용훈

# GPIO (1/3)

## GPIO란?

- GPIO(General Purpose Input Output)
  - 입출력을 포함한 동작이 실행시간 중에 사용자에 의해 제어될 수 있는 집적회로나 전기 회로 기판의 디지털 신호 핀
  - 디지털 입출력을 기본으로 하며 아날로그 데이터와 PWM 또한 처리 가능
  - UART, I2C, SPI 등 특정한 기능을 제공하는 핀이 있다.
- 라즈베리 파이는 40개의 핀을 가지고 있다.
- 3.3V 전원 전압으로 구동

| 논리 레벨 | 전압 범위 |
|---|---|
| 1 / HIGH | 2.7V ~ 3.3V |
| 0 / LOW | 0.0V ~ 0.7V |



# GPIO (2/3)

## WiringPi

- 라즈베리 파이 GPIO를 제어하기 위한 C 기반 라이브러리

```
pi@raspberrypi:~/wiringPi $ gpio readall
 +-----+-----+---------+------+---+---Pi 3B--+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 |     |     |    3.3v |      |   |  1 || 2  |   |      | 5v      |     |     |
 |   2 |   8 |   SDA.1 |   IN | 1 |  3 || 4  |   |      | 5v      |     |     |
 |   3 |   9 |   SCL.1 |   IN | 1 |  5 || 6  |   |      | 0v      |     |     |
 |   4 |   7 | GPIO. 7 |   IN | 1 |  7 || 8  | 0 | IN   | TxD     | 15  | 14  |
 |     |     |      0v |      |   |  9 || 10 | 1 | IN   | RxD     | 16  | 15  |
 |  17 |   0 | GPIO. 0 |   IN | 0 | 11 || 12 | 0 | IN   | GPIO. 1 | 1   | 18  |
 |  27 |   2 | GPIO. 2 |   IN | 0 | 13 || 14 |   |      | 0v      |     |     |
 |  22 |   3 | GPIO. 3 |   IN | 0 | 15 || 16 | 0 | IN   | GPIO. 4 | 4   | 23  |
 |     |     |    3.3v |      |   | 17 || 18 | 0 | IN   | GPIO. 5 | 5   | 24  |
 |  10 |  12 |    MOSI |   IN | 0 | 19 || 20 |   |      | 0v      |     |     |
 |   9 |  13 |    MISO |   IN | 0 | 21 || 22 | 0 | IN   | GPIO. 6 | 6   | 25  |
 |  11 |  14 |    SCLK |   IN | 0 | 23 || 24 | 1 | IN   | CE0     | 10  | 8   |
 |     |     |      0v |      |   | 25 || 26 | 1 | IN   | CE1     | 11  | 7   |
 |   0 |  30 |   SDA.0 |   IN | 1 | 27 || 28 | 1 | IN   | SCL.0   | 31  | 1   |
 |   5 |  21 | GPIO.21 |   IN | 1 | 29 || 30 |   |      | 0v      |     |     |
 |   6 |  22 | GPIO.22 |   IN | 1 | 31 || 32 | 0 | IN   | GPIO.26 | 26  | 12  |
 |  13 |  23 | GPIO.23 |   IN | 0 | 33 || 34 |   |      | 0v      |     |     |
 |  19 |  24 | GPIO.24 |   IN | 0 | 35 || 36 | 0 | IN   | GPIO.27 | 27  | 16  |
 |  26 |  25 | GPIO.25 |   IN | 0 | 37 || 38 | 0 | IN   | GPIO.28 | 28  | 20  |
 |     |     |      0v |      |   | 39 || 40 | 0 | IN   | GPIO.29 | 29  | 21  |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+---Pi 3B--+---+------+---------+-----+-----+
```

```
#ifndef __WIRING_PI_H__
#define __WIRING_PI_H__

// C doesn't have true/false by default and I can never remember which
//      way round they are, so ...

#ifndef TRUE
#  define       TRUE    (1==1)
#  define       FALSE   (!TRUE)
#endif

// Handy defines

// wiringPi modes

#define WPI_MODE_PINS           0
#define WPI_MODE_GPIO           1
#define WPI_MODE_GPIO_SYS       2
#define WPI_MODE_PHYS           3
#define WPI_MODE_PIFACE         4
#define WPI_MODE_UNINITIALISED  -1

// Pin modes

#define INPUT                   0
#define OUTPUT                  1
#define PWM_OUTPUT              2
#define GPIO_CLOCK              3
#define SOFT_PWM_OUTPUT         4
#define SOFT_TONE_OUTPUT        5
#define PWM_TONE_OUTPUT         6

#define LOW                     0
#define HIGH                    1
```

# GPIO (3/3)

## 핀 맵(pin map)

| SENSOR | wPIN | PHYSICAL | |
|---|---|---|---|
| | | 3.3V | 1 |
| I2C Text LCD<br>4-FND | 8 | SDA 1 | 3 |
| | 9 | SCL 1 | 5 |
| Push Switch | 7 | GPIO 7 | 7 |
| | | 0V | 9 |
| Push Switch | 0 | GPIO 0 | 11 |
| PIR | 2 | GPIO 2 | 13 |
| DHT11 | 3 | GPIO 3 | 15 |
| | | 3.3V | 17 |
| SPI<br>RFID Reader<br>MCP3008 | 12 | MOSI | 19 |
| | 13 | MISO | 21 |
| | 14 | SCLK | 23 |
| | | 0V | 25 |
| Active Buzzer | 30 | SDA 0 | 27 |
| Passive Buzzer | 21 | GPIO 21 | 29 |
| Step Motor | 22 | GPIO 22 | 31 |
| | 23 | GPIO 23 | 33 |
| | 24 | GPIO 24 | 35 |
| | 25 | GPIO 25 | 37 |
| | | 0V | 39 |

| PHYSICAL | | wPIN | SENSOR |
|---|---|---|---|
| 2 | 5V | | |
| 4 | 5V | | |
| 6 | 0V | | |
| 8 | TxD | 15 | 4-FND |
| 10 | RxD | 16 | RFID Reader |
| 12 | GPIO 1 | 1 | IR Receiver |
| 14 | 0V | | |
| 16 | GPIO 4 | 4 (Trig) | Ultrasonic |
| 18 | GPIO 5 | 5 (Echo) | |
| 20 | 0V | | |
| 22 | GPIO 6 | 6 | DC Motor |
| 24 | CE0 | 10 | RFID Reader |
| 26 | CE1 | 11 | MCP3008(ADC) |
| 28 | SCL 0 | 31 | DC Motor |
| 30 | 0V | | |
| 32 | GPIO 26 | 26 | Servo Motor |
| 34 | 0V | | |
| 36 | GPIO 27 | 27 | RGB LED Red |
| 38 | GPIO 28 | 28 | RGB LED Green |
| 40 | GPIO 29 | 29 | RGB LED Blue |

# wiringPi (1/3)

## 라이브러리 함수

| int wiringPiSetup(void) | | |
|---|---|---|
| 위 치 | wiringPi / wiringPi.h | |
| 기 능 | GPIO 초기 설정 | |
| 매개변수 | void | 없음 |
| 리턴 값 | int | -1 초기화 실패 |

| void pinMode(int pin, int mode) | | |
|---|---|---|
| 위 치 | wiringPi / wiringPi.h | |
| 기 능 | 특정 핀의 동작모드 설정 | |
| 매개변수 | int pin | 동작 모드 설정할 핀 번호 |
| | int mode | INPUT / OUTPUT<br>PWM_OUTPUT / SOFT_PWM_OUTPUT |
| 리턴 값 | void | 없음 |

| void digitalWrite(int pin, int value) | | |
|---|---|---|
| 위 치 | wiringPi / wiringPi.h | |
| 기 능 | 특정 핀에 LOW 또는 HIGH 신호 출력 | |
| 매개변수 | int pin | 출력할 핀 번호 |
| | int value | 출력 값 LOW / HIGH |
| 리턴 값 | void | 없음 |

| int digitalRead(int pin) | | |
|---|---|---|
| 위 치 | wiringPi / wiringPi.h | |
| 기 능 | 특정 핀으로 입력되는 LOW 또는 HIGH 신호 읽어오기 | |
| 매개변수 | int pin | 입력 신호를 읽어 올 핀 번호 |
| 리턴 값 | int | 입력된 LOW 또는 HIGH 신호 값 |

# wiringPi (2/3)

## 라이브러리 함수

| void analogWrite(int pin, int value) | | |
|---|---|---|
| 위 치 | wiringPi / wiringPi.h | |
| 기 능 | 특정 핀에 아날로그 값 출력 | |
| 매개변수 | int pin | 출력할 핀 번호 |
| | int value | 출력 값 |
| 리턴 값 | void | 없음 |

| void pwmWrite(int pin, int value) | | |
|---|---|---|
| 위 치 | wiringPi / wiringPi.h | |
| 기 능 | 지정된 핀에 지정된 값의 PWM 출력 | |
| 매개변수 | int pin | 출력할 핀 번호 |
| | int value | 0 ~ 1023 범위의 값 출력 |
| 리턴 값 | void | 없음 |

| int analogRead(int pin) | | |
|---|---|---|
| 위 치 | wiringPi / wiringPi.h | |
| 기 능 | 특정 핀으로 입력되는 아날로그 값 읽어오기 | |
| 매개변수 | int pin | 입력 신호를 읽어 올 핀 번호 |
| 리턴 값 | int | 입력된 값 |

| void delay(unsigned int howLong) | | |
|---|---|---|
| 위 치 | wiringPi / wiringPi.h | |
| 기 능 | 지정된 밀리 세컨드만큼 일시 정지 | |
| 매개변수 | unsigned int howLong | 일시 정지할 밀리 세컨드 |
| 리턴 값 | void | 없음 |

# wiringPi (3/3)

## 프로그램 기본 구조

### ▶ 라이브러리 추가하기

```
#include <wiringPi.h>        //- GPIO용 라이브러리 wiringPi
#include <stdio.h>           //- C 프로그램용 표준 입출력 라이브러리
```

### ▶ wiringPi 라이브러리 초기화

```
wiringPiSetup()              //- wiringPi 라이브러리 초기화
```

### ▶ 핀 동작 모드 설정

Raspberry Pi의 GPIO핀 중에서 사용될 핀의 동작 모드 설정

```
- 출력 모드 설정        pinMode(Pin_Num, OUTPUT)
- 입력 모드 설정        pinMode(Pin_Num, INPUT)
```

### ▶ 데이터 입출력

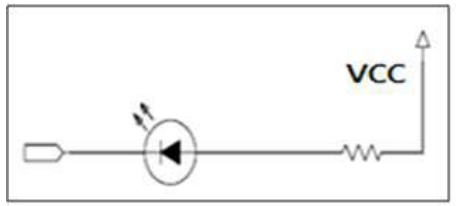
동작 모드가 설정된 핀에 데이터 입출력

```
- 출력      digitalWrite(Pin_Num, value) / analogWrite(Pin_Num, value)
- 입력      digitalRead(Pin_Num)         / analogRead(Pin_Num)
```

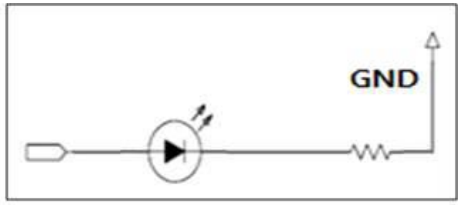
# RGB LED(1/4)

## LED(Light Emitting Diode)

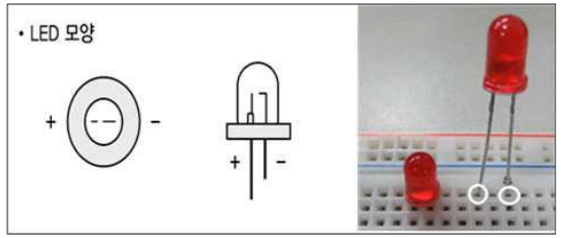
- 발광 다이오드 - 전류를 가하면 빛을 발하는 반도체 소자
- 장치의 상태를 표현하거나 빛을 내는 용도로 사용
- 타입에 따라 에노드(Anode)와 캐소드(Cathode)로 구분
- 키트의 LED는 Red, Green, Blue 빛의 3원소를 이용하여 색 표현

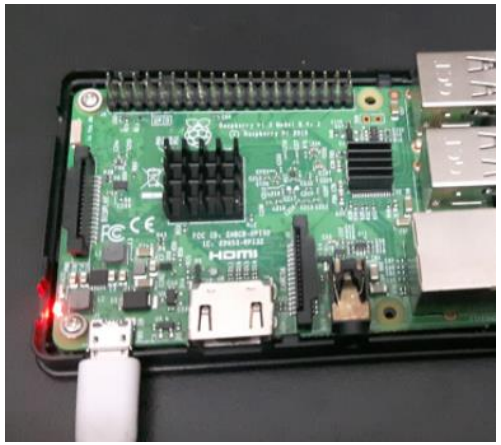



[ Anode Type ]

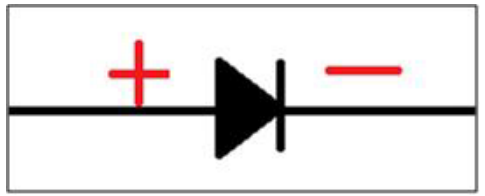



[ Cathode Type ]



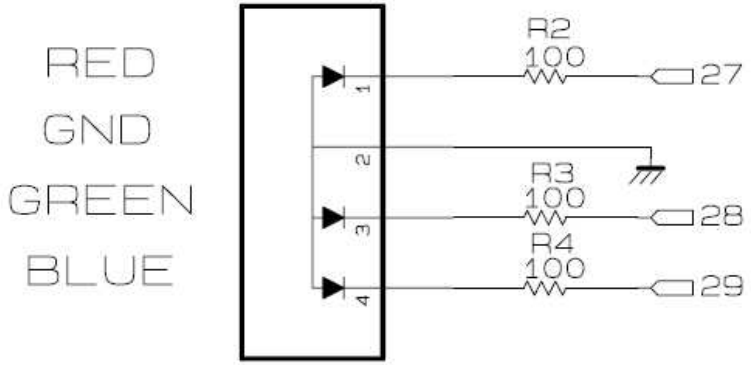
| 연결 방식 | 애노드 | | 캐소드 | |
|---|---|---|---|---|
| 동작 신호 | LED ON | => LOW | LED ON | => HIGH |
| | LED OFF | => HIGH | LED OFF | => LOW |



# RGB LED(2/4)

## LED 연결 핀

키트의 RGB-LED 모듈




| BCM | wPi | Name | Mode | V | Physical | V | Mode | Name | wPi | BCM |
|---|---|---|---|---|---|---|---|---|---|---|
| 6 | 22 | GPIO.22 | IN | 1 | 31 \|\| 32 | 0 | IN | GPIO.26 | 26 | 12 |
| 13 | 23 | GPIO.23 | IN | 0 | 33 \|\| 34 | | | 0v | | |
| 19 | 24 | GPIO.24 | IN | 0 | 35 \|\| 36 | 0 | IN | GPIO.27 | 27 | 16 |
| 26 | 25 | GPIO.25 | IN | 0 | 37 \|\| 38 | 0 | IN | GPIO.28 | 28 | 20 |
| | | 0v | | | 39 \|\| 40 | 0 | IN | GPIO.29 | 29 | 21 |

Pi 3B

GPIO 핀 맵에서 wiringPi 핀 – 27,28,29

# RGB LED(3/4)

## LED 제어 코드

[ EX_RGB_LED.c ]

```
01 //- 센서 제어 라이브러리 ----------------------------------------------------------
02 #include <stdio.h>
03 #include <wiringPi.h>
04
05 //- 센서 연결 핀 선언 ----------------------------------------------------------
06 #define R_LED_PIN                    27    //WPI
07 #define G_LED_PIN                    28    //WPI
08 #define B_LED_PIN                    29    //WPI
09
10 //- 프로그램 실행 함수 ----------------------------------------------------------
11 int main(void){
12
13         //- wiringPi 초기화
14         if( wiringPiSetup() == -1)
15         {
16                 printf("Setup Fail ~!!");
17                 return -1;
18         }
19
20         //- 센서 초기화
21         pinMode(R_LED_PIN,   OUTPUT);
22         pinMode(G_LED_PIN,   OUTPUT);
23         pinMode(B_LED_PIN,   OUTPUT);
24
25         //- 삼색 LED 제어
26         while(1)
27         {
28                 digitalWrite(R_LED_PIN, HIGH) ;
29                 digitalWrite(B_LED_PIN, LOW) ;
30                 printf("RED LED ON ~₩n");
31                 delay(1000);
32                 digitalWrite(R_LED_PIN, LOW) ;
33                 digitalWrite(G_LED_PIN, HIGH) ;
34                 printf("GREEN LED OFF ~₩n");
35                 delay(1000);
36
37                 digitalWrite(G_LED_PIN, LOW) ;
38                 digitalWrite(B_LED_PIN, HIGH) ;
39                 printf("BLUE LED ON ~₩n");
40                 delay(1000);
4 1
42                 digitalWrite(R_LED_PIN, LOW) ;
43                 digitalWrite(G_LED_PIN, LOW) ;
44                 digitalWrite(R_LED_PIN, LOW) ;
45                 printf("RGB LED OFF ~₩n");
46                 delay(2000);
47     }
48     return 0;
49 }
```

# RGB LED(4/4)

## LED 제어 코드

- 코드 빌드
  - 형식: gcc –l[라이브러리 이름] [파일명] –o [실행 파일명]

```
pi@raspberrypi:~/Example/GPIO $ gcc -lwiringPi EX_RGB_LED.c -o RGB_LED
pi@raspberrypi:~/Example/GPIO $ ls
EX_RGB_LED.c  RGB_LED
pi@raspberrypi:~/Example/GPIO $
```

- 실행하기
  - 형식 : ./[실행 파일명]

```
pi@raspberrypi:~/Example/GPIO $ ./RGB_LED
Blue LED ON ~
Green LED OFF ~
Red LED ON ~
```

- 실행 종료하기

```
Red LED OFF ~
Blue LED ON ~
Green LED OFF ~
Red LED ON ~
Red LED OFF ~
^C
pi@raspberrypi:~/Example/GPIO $
```

실행 종료: Ctrl + C

# Active Buzzer(1/3)

## Buzzer

- 금속판을 떨리게 해서 소리를 출력하는 부품으로 단순음만 출력

| | **능동 부저 (Active Buzzer)** | 수동 부저 (Passive Buzzer) |
|---|---|---|
| 특징 | -내장된 회로 존재<br>-전원만 인가해주면 소리 출력 | -내장 회로 없음<br>-출력 주파수 조절로 소리 출력 |
| 용도 | -음성 출력용 불가능<br>-전자기기 내부 음향 신호출력용 | -전자시계 알람 멜로디<br>-전화기의 벨소리 |
| 모양 | -스티커로 양극/음극 표시<br>-하부에 고무로 마무리 | -하부를 PCB형태로 기판이 보임 |

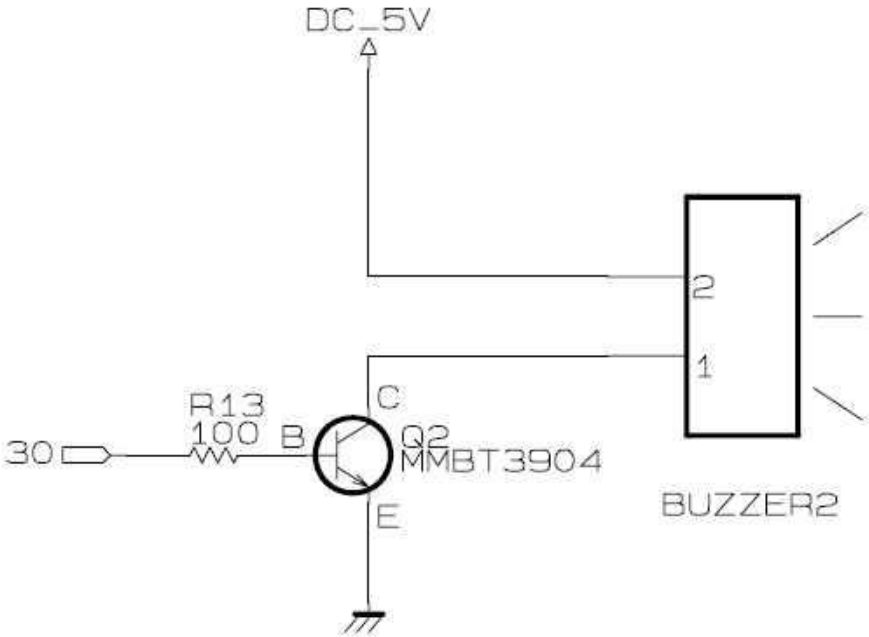
# Active Buzzer(2/3)

## Active Buzzer 연결 핀

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 0v | | | 25 | 26 | 1 | IN | CE1 | 11 | 7 |
| 0 | 30 | SDA.0 | IN | 1 | 27 | 28 | 1 | IN | SCL.0 | 31 | 1 |
| 5 | 21 | GPIO.21 | IN | 1 | 29 | 30 | | | 0v | | |
| 6 | 22 | GPIO.22 | IN | 1 | 31 | 32 | 0 | IN | GPIO.26 | 26 | 12 |
| 13 | 23 | GPIO.23 | IN | 0 | 33 | 34 | | | 0v | | |
| 19 | 24 | GPIO.24 | IN | 0 | 35 | 36 | 0 | IN | GPIO.27 | 27 | 16 |
| 26 | 25 | GPIO.25 | IN | 0 | 37 | 38 | 0 | IN | GPIO.28 | 28 | 20 |
| | | 0v | | | 39 | 40 | 0 | IN | GPIO.29 | 29 | 21 |
| BCM | wPi | Name | Mode | V | Physical | | V | Mode | Name | wPi | BCM |

Pi 3B

GPIO 핀 맵에서 wiringPi 핀 – 30




Active Buzzer 회로 구조

# Active Buzzer(3/3)

[ EX_A_Buzzer.c ]

```
//- 관련 라이브러리 추가----------------------------------------------------------------
#include <wiringPi.h>
#include <stdlib.h>

//- 핀 번호 선언 ----------------------------------------------------------------------
#define BUZ_A_PIN                30          //- wiringPi

//- Buzzer 제어 기능 구현 -------------------------------------------------------------
int main(void)
{
    //- wiringPi 초기화
    if( wiringPiSetup() == -1)
    {cl
            printf("Setup Fail ~!!");
            return -1;
    }
    //- 센서 초기화
    pinMode(BUZ_A_PIN, OUTPUT);

    while(1)
    {
            digitalWrite(BUZ_A_PIN, HIGH);
            delay(1000);

            digitalWrite(BUZ_A_PIN, LOW);
            delay(1000);
    }
}
```

코드 빌드

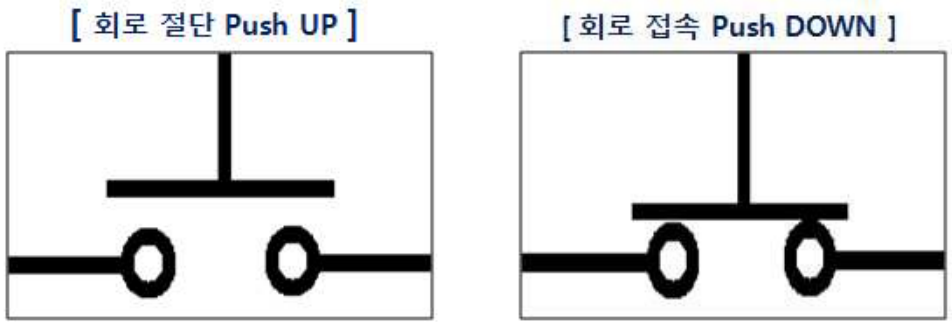
형식: gcc -lwiringPi [파일명] -o [실행 파일명]

실행을 중지할 때는 부저가 꺼진 상태에서 Ctrl + C

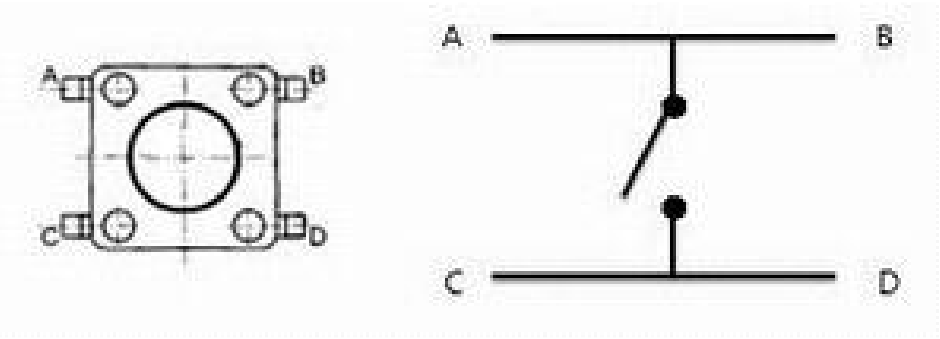
# Push Switch(1/3)

## Push Switch

- 버튼을 눌러 회로를 절단 또는 접속시키는 스위치




- 회로가 연결되면 저류가 흘러서 지나갈 수 있고,
  회로가 끊겨 있다면 전류가 흘러가지 못한다.




스위치의 회로 구조

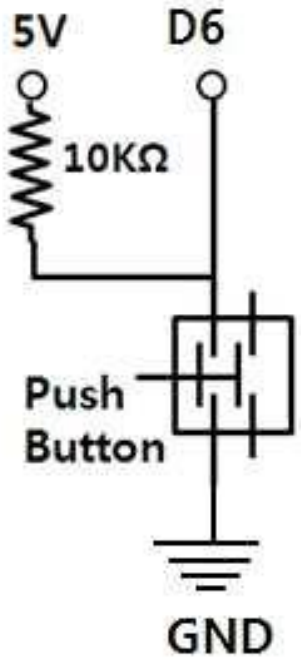
# Push Switch(2/3)

## 방식에 따른 Push Switch 연결 구조

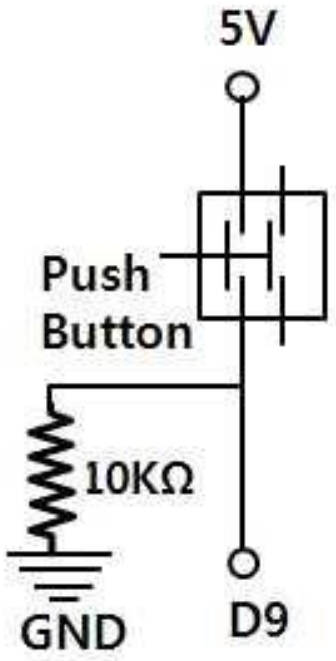
- 플로팅(floating): 전류가 흐르지 않는 상태임에도 해당 스위치에 전류 흐름 여부를 알 수 없는 상태
- 저항은 해당 현상을 방지하기 위한 장치
- 저항의 위치에 따라 풀 업과 풀 다운으로 구분한다.

| 풀업(Pull Up) 방식 | | |
|---|---|---|
| 연결 | 저 항 | 5V와 입력 핀 |
| | 푸시버튼 | 접지 GND |
| 동작 | 입력 O | LOW(OFF) |
| | 입력 X | **HIGH(ON)** |

| 풀다운(Pull Down) 방식 | | |
|---|---|---|
| 연결 | 저 항 | GND와 입력 핀 |
| | 푸시버튼 | 전원 5V |
| 동작 | 입력 O | HIGH(ON) |
| | 입력 X | **LOW(OFF)** |




Pull up




Pull down

# Push Switch(3/3)

```
#include <stdio.h>
#include <stdlib.h>
#include <wiringPi.h>
#define PUSHSWL 0
#define PUSHSWR 7

int main(){

    //wiringPi 초기화
    if(wiringPiSetup()==-1){
        printf("Set up fail~!");
        return -1;
    }

    pinMode(PUSHSWL,INPUT);
    pinMode(PUSHSWR,INPUT);

    while(1){
        if(digitalRead(PUSHSWL)==LOW){
            printf("Left Switch Pushed!\n");
            delay(300);
        }else if(digitalRead(PUSHSWR)==LOW){
            printf("Right Switch Pushed!\n");
            delay(300);
        }
    }
    return 0;
}
```

# 과제 1

## Switch를 누를 때마다 LED의 상태가 변하는 프로그램

- 프로그램이 실행되면 LED는 소등 상태이다.
- 소등 상태에서 Switch를 누르면 녹색 LED가 점등된다.
- 녹색 LED가 켜진 상태에서 Switch를 누르면 빨간색 LED가 점등된다.
- 빨간색 LED가 켜진 상태에서 Switch를 누르면 파란색 LED가 점등된다.
- 파란색 LED가 켜진 상태에서 Switch를 누르면 LED가 소등된다.

# 과제 2

## Switch를 누르면 부저가 울리는 프로그램

- 프로그램을 실행하면 능동 부저가 울리지 않는 상태이다.
- 왼쪽버튼을 누르면 부저가 딜레이(delay)없이 울린다.
- 부저가 울릴 때 오른쪽 버튼을 누르면 부저가 꺼진다.

# 고생하셨습니다

Thank you!